Тема 5. Работа с файловой системой - асинхронная

хекслет колледж

Цель занятия:

Изучить способы неблокирующего взаимодействия с файловой системой. Понять эволюцию подходов в JavaScript: от функций обратного вызова (callbacks) до современного стандарта async/await.

хекслет колледж

Учебные вопросы:

1. Концепция Event Loop
2. Callback-style
2. Модуль fs.promises.
3. Async/Await: Современный и рекомендуемый синтаксис.
4. Когда использовать синхронные методы, а когда - асинхронные.

хекслет колледж

1. Концепция Event Loop и неблокирующий ввод-вывод (I/O)

В чем проблема блокировки?

В большинстве традиционных языков программирования выполнение кода приостанавливается, пока операционная система читает данные с диска.

Если файл большой, поток (Thread) «спит».

В веб-серверах это решается созданием нового потока на каждый запрос, что потребляет много оперативной памяти.

кекслет колледж

Подход Node.js: «Сделай и позови, когда закончишь»

Node.js - однопоточная среда. Мы не можем позволить себе «спать» во время чтения файла, иначе весь сервер зависнет для всех пользователей.

Как это работает на практике:

- Node.js отправляет запрос к файловой системе (через внутреннюю библиотеку libuv).
- Не дожидаясь ответа, Node.js переходит к выполнению следующей строки кода.
- Когда файл прочитан, событие попадает в очередь, и Event Loop (событийный цикл) передает данные в наш callback или разрешает Promise.

хекслет колледж

Механика Event Loop (кратко)

Event Loop - это бесконечный цикл, который проверяет:

- Есть ли задачи в Call Stack (выполняется ли сейчас какой-то код?).
- Если стек пуст, он заглядывает в Callback Queue (очередь задач) и переносит готовую задачу (например, «файл прочитан») в стек на выполнение.

хекслет колледж

Синхронно vs Асинхронно (Пример-сравнение)

Синхронный подход (уже изучен):

```
console.log('1. Начинаю');
const data = fs.readFileSync(filePath, 'utf8'); // Зависли тут
console.log('2. Прочитал:', data);
console.log('3. Конец');
```

Порядок в консоли: 1 → 2 → 3.

кекслет колледж

Асинхронный подход (новый):

```
console.log('1. Начинаю');
fs.readFile(filePath, 'utf8', (err, data) => {
    console.log('2. Прочитал:', data); // Выполнится позже
});
console.log('3. Конец');
```

Порядок в консоли: 1 → 3 → 2.

хекслет колледж

Ключевые термины для запоминания:

- Non-blocking I/O: Ввод-вывод, который не заставляет программу ждать.
- Call Stack: Место, где выполняется текущий код.
- Callback Queue: Очередь из «завершенных» асинхронных дел, которые ждут своего часа.
- libuv: Библиотека на языке C++, которая фактически выполняет чтение файлов в фоновых потоках, пока основной поток JS занят.

хекслет колледж

Вывод:

Асинхронность в Node.js - это не «параллельное выполнение кода», это умение грамотно перекладывать тяжелые задачи (I/O) на плечи операционной системы, оставаясь свободным для обработки новых команд.

хекслет колледж

2. Callback-style

Что такое Callback (колбэк)?

В Node.js callback - это функция, которая передается в качестве аргумента в асинхронный метод.

Она «вызывается обратно», когда операция ввода-вывода (I/O) завершена.

хекслет колледж

Правило Error-First Callback

В Node.js принят стандарт: первым аргументом в колбэк-функцию всегда передается объект ошибки (err), а вторым - результат операции (data).

- Если err не равен null, значит, что-то пошло не так.
- Если err равен null, значит, данные успешно получены.

хекслет колледж

Пример чтения файла:

```
const fs = require('fs');
const path = require('path');

const filePath = path.join(__dirname, 'test.txt');

fs.readFile(filePath, 'utf8', (err, data) => {
    if (err) {
        console.error('Ошибка чтения:', err.message);
        return;
    }
    console.log('Содержимое файла:', data);
});
console.log('Код после readFile продолжает работать...');
```

кекслет колледж

Проблема Callback Hell (Ад колбэков)

Когда нам нужно выполнить несколько асинхронных действий последовательно (например: прочитать файл → изменить данные → сохранить в другой файл → удалить оригинал), код начинает расти вправо, образуя «пирамиду» или «лесенку».

хекслет колледж

Пример "лапшичного" кода:

```
fs.readFile(sourcePath, 'utf8', (err, data) => {
    if (!err) {
        const newData = data.toUpperCase();
        fs.writeFile(destPath, newData, (err) => {
            if (!err) {
                fs.unlink(sourcePath, (err) => {
                    if (!err) {
                        console.log('Все операции завершены успешно!');
                    }
                });
            }
        });
    }
});
```

Минусы такого подхода:
- Трудность чтения: Глазу сложно следить за вложенностью.
- Сложность обработки ошибок: Нужно писать if (err) на каждом уровне.
- Трудность поддержки: Очень легко запутаться в закрывающих скобках });.

кекслет колледж

Основные асинхронные методы (Callback-версии)
Методы те же, что и в синхронной лекции, но без суффикса Sync и с обязательным колбэком в конце:

fs.readFile(path, encoding, callback) - чтение.
fs.writeFile(path, data, options, callback) - запись.
fs.appendFile(path, data, callback) - добавление.
fs.unlink(path, callback) - удаление.
fs.mkdir(path, options, callback) - создание папки.

хекслет колледж

- Итог:
  Не блокируем поток: Пока файл читается, Node.js может обрабатывать другие задачи.
- Проверяем err первым: Это признак хорошего тона и стабильного кода.
- Избегаем глубокой вложенности: Если ваш код уходит далеко вправо - пора переходить на Promises или Async/Await.
- Callback-style: **Старый** стандарт работы с fs.

хекслет колледж

3. Promises (Промисы): Модуль fs.promises.

Promise (обещание) - это объект, представляющий результат асинхронной операции.
Вместо того чтобы передавать колбэк «внутрь» функции, функция возвращает объект-обещание, с которым мы работаем «снаружи».

хекслет колледж

Промис может находиться в одном из 3 состояний:

- Pending (Ожидание) - операция еще идет.
- Fulfilled (Выполнено) - всё прошло успешно, данные получены.
- Rejected (Отклонено) - произошла ошибка.

хекслет колледж

Модуль fs.promises

В Node.js есть специальный подмодуль, где все методы файловой системы возвращают промисы вместо использования колбэков.

Как подключить:

```
const fs = require('fs').promises;
```

хекслет колледж

Обработка через .then() и .catch()

Этот синтаксис позволяет избавиться от вложенности (Callback Hell) и выстроить логику в цепочку.

- .then() - вызывается, если операция завершилась успешно. Принимает данные.
- .catch() - вызывается, если произошла любая ошибка на любом этапе цепочки.

хекслет колледж

Пример чтения файла:

```
const fs = require('fs').promises;
const path = require('path');

const filePath = path.join(__dirname, 'test.txt');

fs.readFile(filePath, 'utf8')
    .then((data) => {
        console.log('Данные получены:', data);
    })
    .catch((err) => {
        console.error('Ошибка при чтении:', err.message);
    });
```

кекслет колледж

Цепочки промисов (Chaining)

Главное преимущество промисов - возможность передавать результат одной операции следующей.
Больше никакой «лесенки»!

хекслет колледж

Пример: прочитать → изменить → записать:

```
const fs = require('fs').promises;
const path = require('path');

const source = path.join(__dirname, 'source.txt');
const target = path.join(__dirname, 'target.txt');

fs.readFile(source, 'utf8')
    .then((data) => {
        const processed = data.toUpperCase();
        console.log('1. Файл прочитан и обработан');
        return fs.writeFile(target, processed); // Возвращаем новый промис
    })
    .then(() => {
        console.log('2. Результат записан в новый файл');
    })
    .catch((err) => {
        // Одинаково поймает ошибку и на чтении, и на записи
        console.error('Произошла ошибка в цепочке:', err.message);
    });
```

кекслет колледж

Преимущества перед колбэками:

- Линейная структура: Код растет вниз, а не вправо.
- Единый центр обработки ошибок: Один .catch() в конце цепочки заменяет десятки проверок if (err) внутри каждого колбэка.
- Комбинирование: Можно запустить несколько операций одновременно через Promise.all().

хекслет колледж

4. Async/Await: Современный и рекомендуемый синтаксис.

Async/Await - это специальный синтаксис (синтаксический сахар) для работы с промисами.
Он не меняет того, как работает Node.js под капотом, но делает код максимально чистым.

хекслет колледж

async - ключевое слово перед функцией. Оно говорит о том, что внутри функции будут асинхронные операции, и сама функция теперь всегда возвращает промис.

await - ключевое слово, которое можно использовать только внутри async-функции. Оно заставляет Node.js подождать выполнения промиса, не блокируя при этом весь поток (Event Loop).

хекслет колледж

Пример:

```
const fs = require('fs').promises;
const path = require('path');

async function readFileContent() {
    const filePath = path.join(__dirname, 'info.txt');
    // Вместо .then() мы просто "ждем" результат в переменную
    const data = await fs.readFile(filePath, 'utf8');
    console.log(data);
}

readFileContent();
```

хекслет колледж

Обработка ошибок через try...catch

В отличие от промисов с их .catch(), при использовании async/await мы возвращаемся к классической конструкции try...catch.

Это удобно, так как один блок может обрабатывать и асинхронные ошибки (чтение файла), и синхронные (ошибки парсинга данных)

хекслет колледж

Пример:

```
async function safeRead() {
    try {
        const data = await fs.readFile('info.txt', 'utf8');
        const json = JSON.parse(data); // Если тут будет ошибка, catch её поймает
        console.log(json);
    } catch (err) {
        console.error('Произошла ошибка:', err.message);
    }
}
```

кекслет колледж

Последовательные операции

С async/await решение задачи «прочитать → изменить → записать» становится линейным и понятным даже новичку.

хекслет колледж

Пример:

```
async function processFile() {
    const source = path.join(__dirname, 'source.txt');
    const target = path.join(__dirname, 'target.txt');

    try {
        // 1. Читаем
        const data = await fs.readFile(source, 'utf8');
        // 2. Обрабатываем (в памяти)
        const upperData = data.toUpperCase();
        // 3. Записываем
        await fs.writeFile(target, upperData);
        console.log('Файл успешно обработан и сохранен!');
    } catch (err) {
        console.error('Ошибка в процессе:', err.message);
    }
}

processFile();
```

хекслет колледж

Важные нюансы:

- Не забывайте async: Если вы напишете await внутри обычной функции, Node.js выдаст синтаксическую ошибку.

- await не делает код синхронным: Пока выполняется await, Event Loop свободен и может обрабатывать другие запросы. Это «умное ожидание».

хекслет колледж

Сравнение трех подходов:

Синтаксис	Пример кода	Читаемость
Callback	fs.readFile(p, (err, d) => { ... })	Плохо (лесенка)
Promises	fs.readFile(p).then(d => { ... })	Средне (цепочки)
Async/Await	const d = await fs.readFile(p);	Отлично (линейно)

кекслет колледж

5. Когда использовать синхронные методы, а когда - асинхронные?

Существуют две крайности:

- использование синхронных методов везде, потому что это "проще"

- использование асинхронных методов там, где в этом нет никакого смысла.

хекслет колледж

Золотое правило Node.js

«Никогда не блокируй Event Loop в коде сервера».

Если ваше приложение должно отвечать на запросы пользователей (веб-сервер, чат-бот, API), любое использование синхронных методов ввода-вывода (I/O) - это потенциальная катастрофа, так как один пользователь "подвесит" сервис для всех остальных.

хекслет колледж

Когда использовать Синхронные методы?

Синхронные методы допустимы только в тех случаях, когда программа не может продолжать работу без этих данных, и при этом нет конкурентных запросов.

Инициализация (Startup): Чтение файлов конфигурации (config.json) при запуске сервера. Если конфиг не прочитан, сервер всё равно не должен запускаться.

CLI-утилиты (Скрипты): Простые консольные инструменты для личного пользования (например, скрипт, который один раз переименовывает файлы в папке).

Тесты: В коде тестов часто используют синхронные методы для простоты настройки окружения перед проверками.

хекслет колледж

Когда использовать Асинхронные методы?

Асинхронные методы обязательны во всех сценариях, где важна производительность и отзывчивость.
Веб-серверы (Express и др.):

- Любая работа с файлами внутри обработчиков запросов (чтение шаблонов, загрузка аватарок, запись логов).
- Высоконагруженные системы: Когда нужно выполнять несколько операций одновременно (например, прочитать 10 разных файлов параллельно).
- Стриминг и видео: Работа с потоковыми данными.

хекслет колледж

Выводы:

- Главная фишка Node.js - неблокирующий ввод-вывод.
- Event Loop (Событийный цикл) - диспетчер, который следит за тем, чтобы вовремя выполнить код из колбэка или промиса, когда данные станут доступны.
- Эволюция методов:

- Callbacks: Фундамент, но порождают «ад колбэков».
- Promises: Позволяют строить цепочки .then().catch(), делая код линейным.
- Async/Await: «Золотой стандарт» современности. Код выглядит как синхронный, но работает асинхронно.

- try...catch: В async/await этот блок снова становится основным инструментом обработки ошибок, как и в синхронном коде.
- Синхронность уместна редко: Только при старте сервера (чтение конфига) или в простых консольных скриптах. В остальное время - только асинхронность.

кекслет колледж

Контрольные вопросы

- Объясните своими словами, что делает Node.js, когда встречает функцию fs.readFile()?
- Почему правило "Error-First Callback" так важно для старого стиля Node.js?
- В чем преимущество использования fs.promises по сравнению с обычным fs?
- Что произойдет с сервером на Express, если внутри обработчика запроса вызвать fs.readFileSync() для файла размером 1 ГБ?
- Зачем нужно ключевое слово await и что будет, если его забыть перед fs.readFile внутри async функции?
- В какой ситуации использование синхронного метода fs.existsSync() предпочтительнее асинхронной проверки?

кекслет колледж